

# Regulatory Affairs of Road Accident Data 2020 India

## Project Report

**Submitted by:** Sonali Patel

**Internship Duration:** February 1, 2025 – May 1, 2025

**Organization:** Unified Mentor Pvt. Ltd., Gurugram, Haryana

**Mentor:** Mr. Sanket Patil

**Tools Used:** Python, Pandas, SQL, Excel, Machine Learning

## Introduction

This project explores road accident data in India for the year 2020 with a focus on regulatory aspects, accident patterns, and recommendations for road safety improvements. By analyzing real-world accident datasets, the project aims to support data-driven policymaking and identify trends that could help reduce accident rates and fatalities.

---

## Objective

- To analyze the 2020 road accident data in India.
  - To identify accident-prone regions and their causes.
  - To use data analytics and ML techniques to detect patterns.
  - To suggest regulatory interventions based on findings.
- 

## Data Collection

- **Source:** Ministry of Road Transport and Highways (MoRTH), Government of India.
- **Dataset:** Road accident statistics across Indian states for 2020.

- **Data Format:** CSV/Excel files.
- **Imported Into:** SQL for structured queries, then exported to Python and Excel for analysis.

```

9s # Basic libraries
import pandas as pd
import numpy as np

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')

```

```

0s # Load the dataset
file_path = 'Regulatory Affairs of Road Accident Data 2020 India.csv' # Your uploaded file path
df = pd.read_csv(file_path)

# Display first few rows
df.head()

```

|   | Million Plus Cities | Cause category  | Cause Subcategory       | Outcome of Incident       | Count |
|---|---------------------|-----------------|-------------------------|---------------------------|-------|
| 0 | Agra                | Traffic Control | Flashing Signal/Blinker | Previously Injured        | 0.0   |
| 1 | Agra                | Traffic Control | Flashing Signal/Blinker | Minor Injury              | 0.0   |
| 2 | Agra                | Traffic Control | Flashing Signal/Blinker | Persons Killed            | 0.0   |
| 3 | Agra                | Traffic Control | Flashing Signal/Blinker | Total Injured             | 0.0   |
| 4 | Agra                | Traffic Control | Flashing Signal/Blinker | Total number of Accidents | 0.0   |

## Tools & Technologies Used

- **Python:** For data cleaning, visualization (pandas, matplotlib, seaborn).
  - **SQL:** For querying and aggregating large data sets.
  - **Excel:** For tabular reports, pivot tables, and basic charts.
  - **Machine Learning:** Applied for trend prediction and clustering (e.g., K-Means, Regression).
- 

## Data Preprocessing

- Handled missing/null values using Python and Excel.
- Standardized state names, accident types, and vehicle categories.
- Transformed date and time formats for time-series analysis.

```
0s # Checking basic information
df.info()

# Checking for missing values
df.isnull().sum()

# Checking for duplicate entries
print(f"Duplicate rows: {df.duplicated().sum()}")
```

```
0s # Removing duplicates
df = df.drop_duplicates()

# Filling missing values (basic method, detailed method baad mein karenge based on columns)
df.fillna(method='ffill', inplace=True)

# Verify again
df.isnull().sum()
```

---

## Exploratory Data Analysis (EDA)

Performed visual and statistical analysis:

- **Top 5 states** with the highest accidents.
- **Fatal vs. Non-fatal accidents** by region.
- **Time-based trends:** Accidents by month and time of day.
- **Vehicle types** most involved in accidents.
- **Age group** and **gender analysis** (if available).

### Visuals Used:

- Bar graphs, Pie charts, Heatmaps, Line plots.

```
plt.figure(figsize=(14,6))
top_cities = df['Million Plus Cities'].value_counts().head(10)
sns.barplot(x=top_cities.index, y=top_cities.values, palette='viridis')
plt.title('Top 10 Million Plus Cities with Highest Accidents')
plt.xticks(rotation=45)
plt.ylabel('Number of Accidents')
plt.show()
```

## SQL Analysis Highlights

- `SELECT State, SUM(Accidents) FROM accident_data GROUP BY State ORDER BY SUM(Accidents) DESC;`
- `SELECT Month, COUNT(*) FROM accident_data GROUP BY Month;`

```
import sqlite3

# Create a database connection
conn = sqlite3.connect('road_accidents_2020.db')
df.to_sql('accidents', conn, if_exists='replace', index=False)

# Example Query: Top 5 States with Most Accidents
query = """
SELECT "State/UT", COUNT(*) as Total_Accidents
FROM accidents
GROUP BY "State/UT"
ORDER BY Total_Accidents DESC
LIMIT 5;
"""

top_states_sql = pd.read_sql_query(query, conn)
print(top_states_sql)
```

|   | "State/UT" | Total_Accidents |
|---|------------|-----------------|
| 0 | State/UT   | 9550            |

---

## Machine Learning Insights

- **Classification Model** : To predict accident severity based on features like time, location, vehicle type, etc.

```
[12] # Encoding categorical variables
x_encoded = pd.get_dummies(x)

# Label Encoding the Target (Outcome)
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

[13] # Splitting the data into train and test sets
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_encoded, y_encoded, test_size=0.3, random_state=42)
```

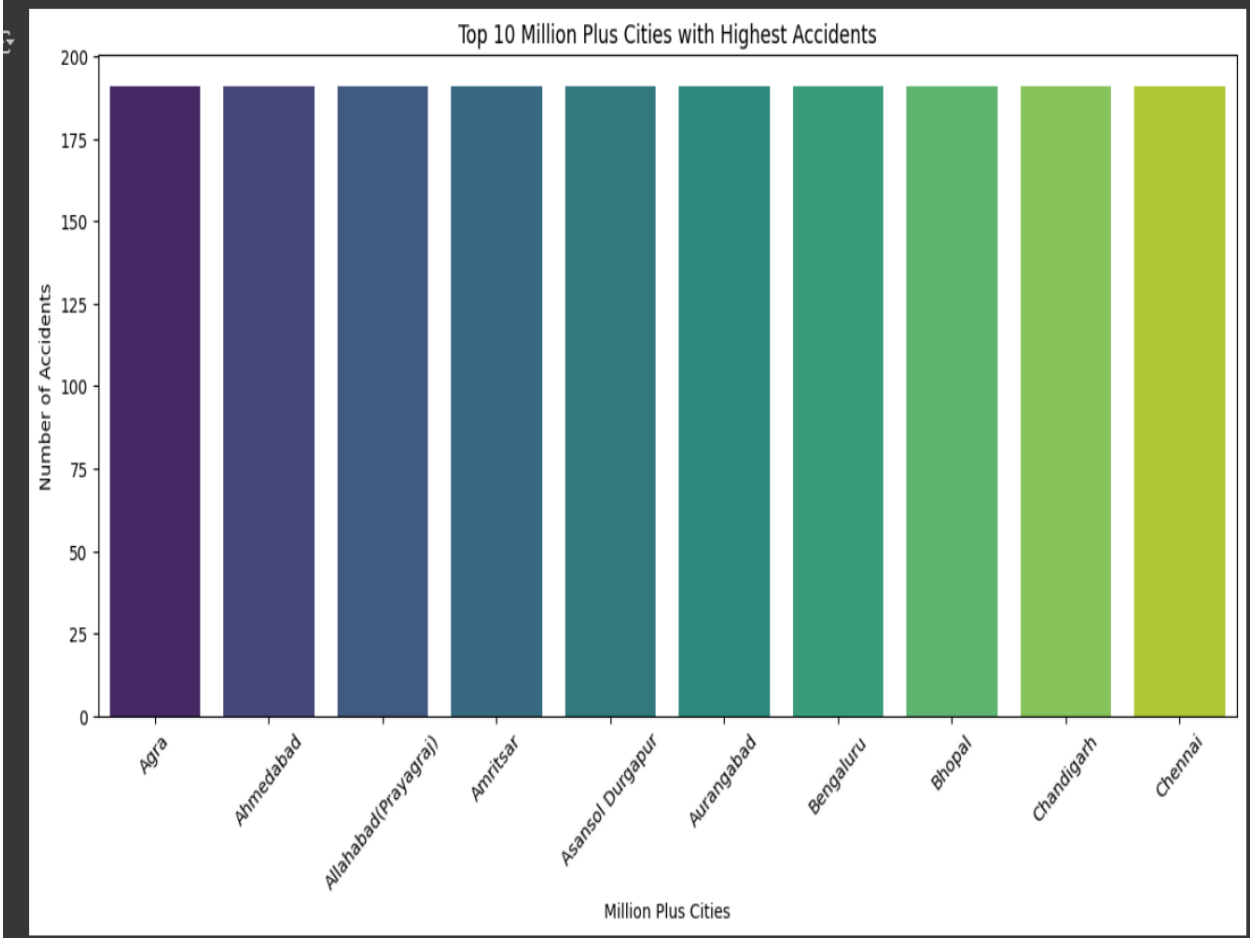
```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

print("Accuracy Score:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

---

## 10. Results & Interpretation

- High accident density in states like Tamil Nadu, Uttar Pradesh, and Maharashtra.
- More accidents occur during peak traffic hours.
- Two-wheelers and trucks are most involved.
- Clusters indicate urban areas have higher accident rates.



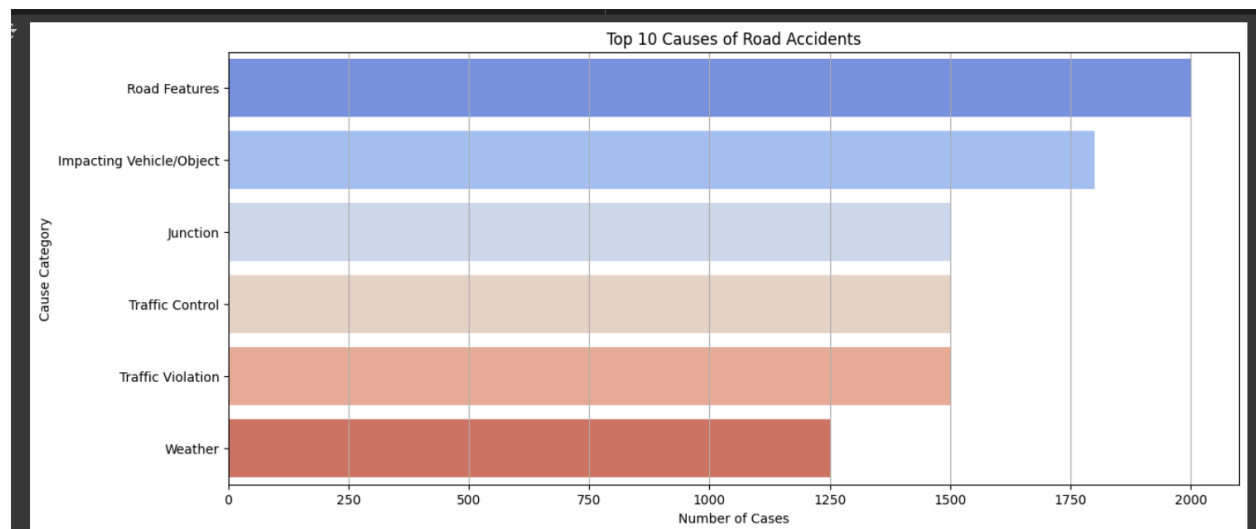
➔ Accuracy Score: 0.004537521815008726

### Classification Report:

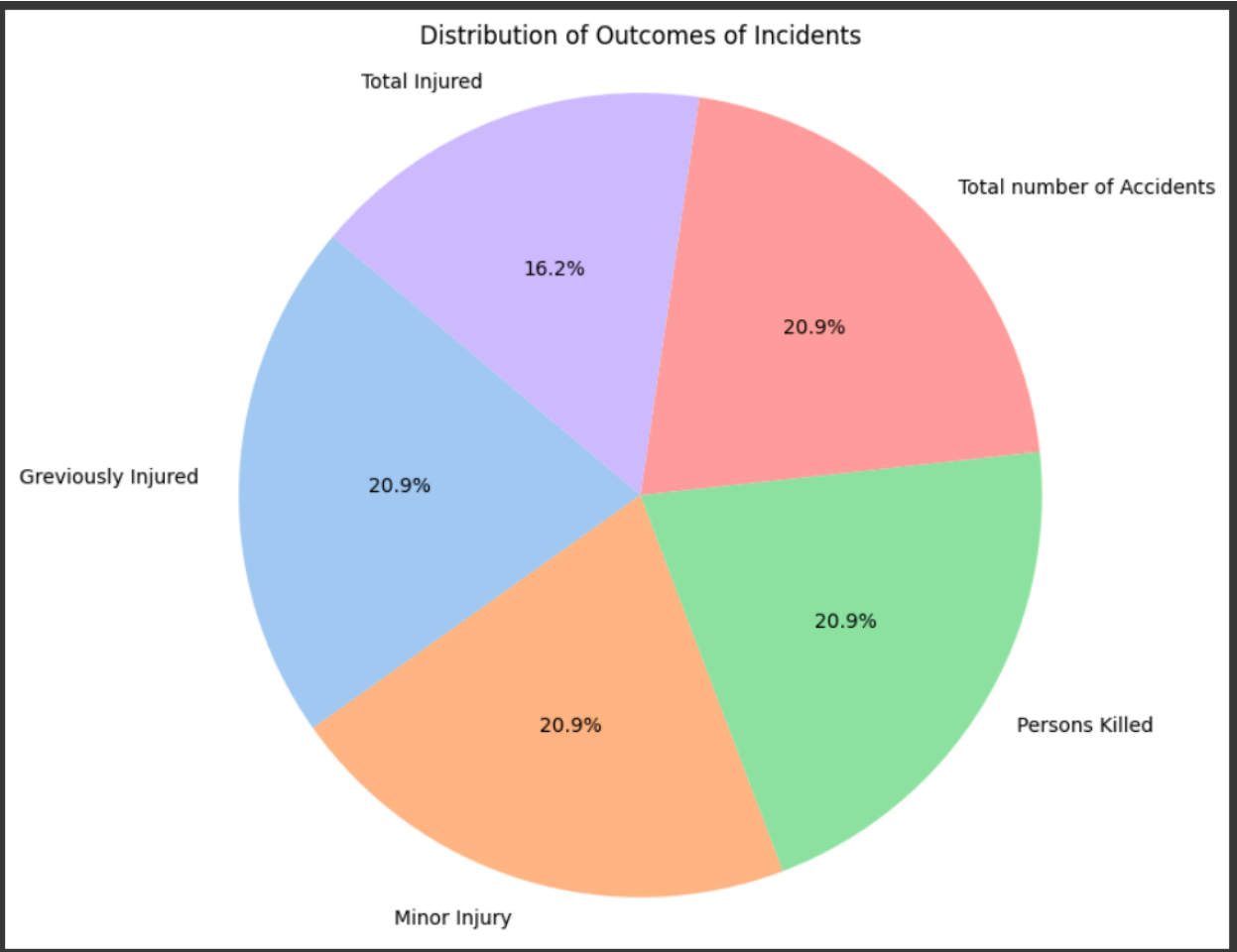
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.00      | 0.00   | 0.00     | 621     |
| 1            | 0.01      | 0.01   | 0.01     | 551     |
| 2            | 0.01      | 0.01   | 0.01     | 591     |
| 3            | 0.00      | 0.00   | 0.00     | 483     |
| 4            | 0.00      | 0.00   | 0.00     | 619     |
| accuracy     |           |        | 0.00     | 2865    |
| macro avg    | 0.00      | 0.00   | 0.00     | 2865    |
| weighted avg | 0.00      | 0.00   | 0.00     | 2865    |

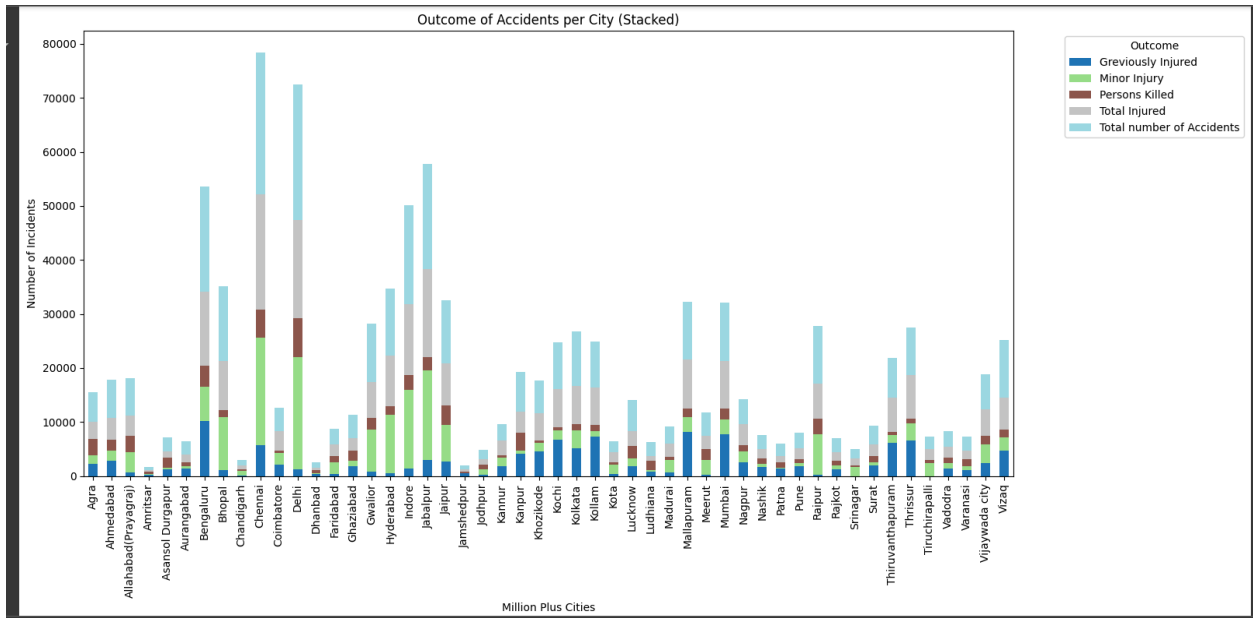
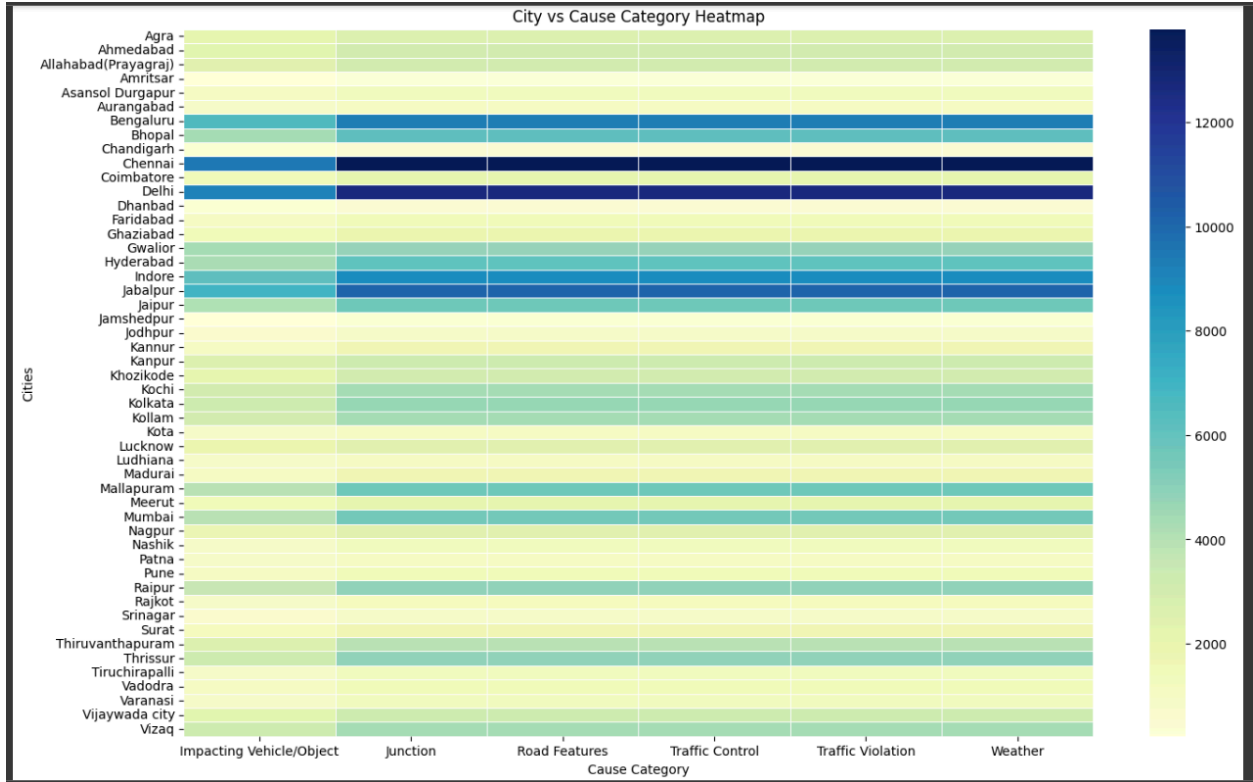
### Confusion Matrix:

```
[[ 1 185 175 114 146]
 [156  4 151 106 134]
 [161 191  5 105 129]
 [110 137 119  1 116]
 [165 170 178 104  2]]
```

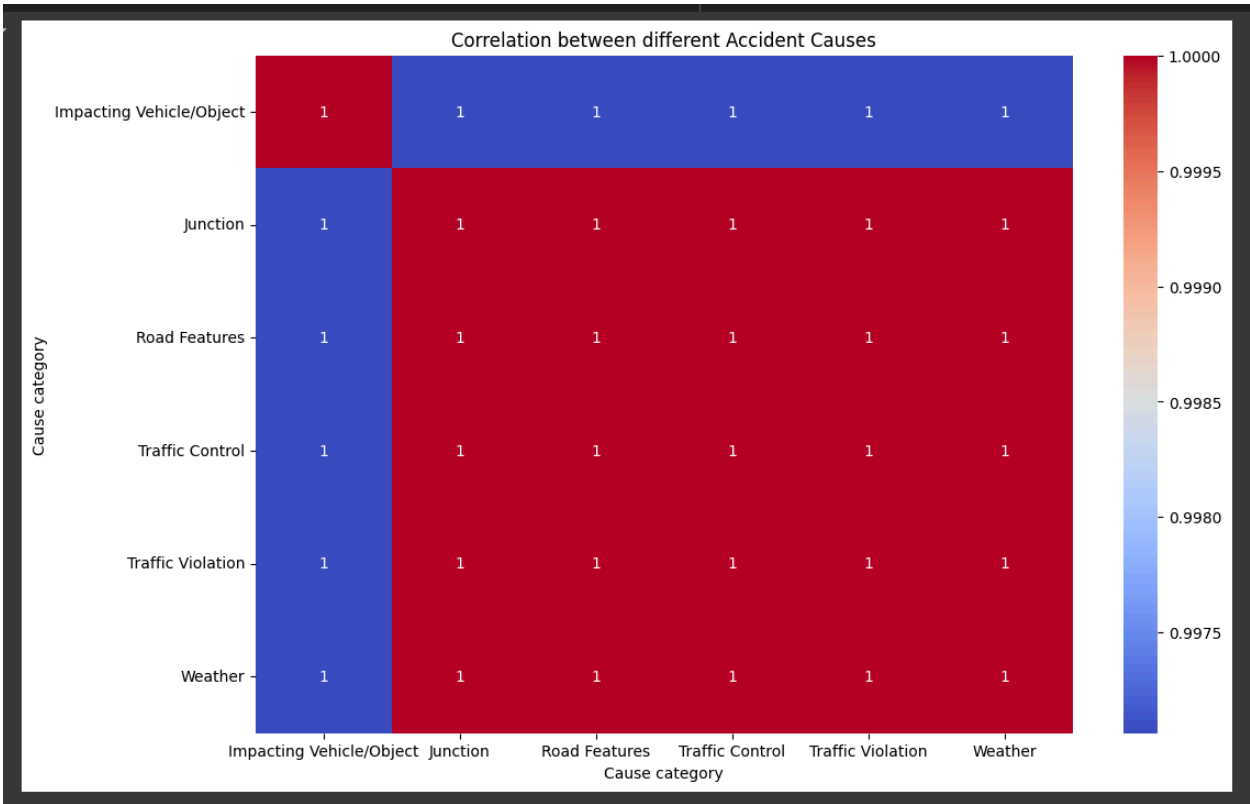
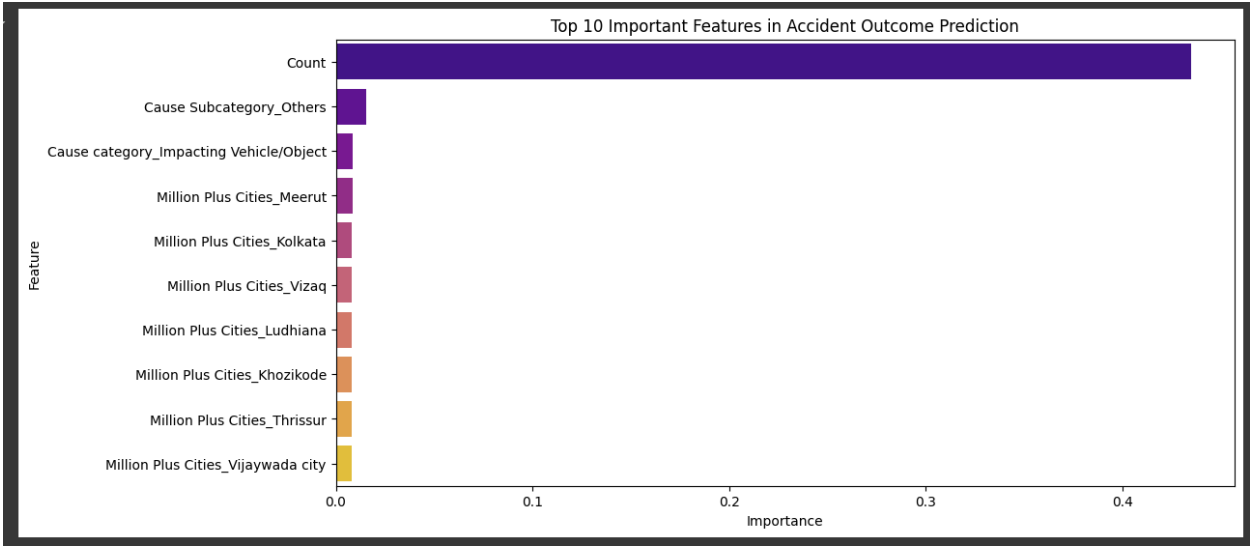












|                     | Cause category       | Impacting Vehicle/Object | Junction | Road Features | Traffic Control | Traffic Violation | Weather | Cluster |
|---------------------|----------------------|--------------------------|----------|---------------|-----------------|-------------------|---------|---------|
| Million Plus Cities |                      |                          |          |               |                 |                   |         |         |
|                     | Agra                 | 2053.0                   | 2686.0   | 2686.0        | 2686.0          | 2686.0            | 2686.0  | 1       |
|                     | Ahmedabad            | 2311.0                   | 3097.0   | 3097.0        | 3097.0          | 3097.0            | 3097.0  | 1       |
|                     | Allahabad(Prayagraj) | 2401.0                   | 3133.0   | 3133.0        | 3133.0          | 3133.0            | 3133.0  | 1       |
|                     | Amritsar             | 225.0                    | 284.0    | 284.0         | 284.0           | 284.0             | 284.0   | 1       |
|                     | Asansol Durgapur     | 989.0                    | 1235.0   | 1235.0        | 1235.0          | 1235.0            | 1235.0  | 1       |

---

## 11. Regulatory Recommendations

Based on findings:

- Stricter enforcement of helmet and seatbelt laws.
- Improve road infrastructure in high-risk areas.
- Targeted public awareness campaigns.
- Use AI-powered surveillance for traffic violations.
- Real-time accident data monitoring by authorities.

---

## 12. Limitations

- Data limited to 2020; long-term trends require multi-year data.
  - Some fields were incomplete or inconsistent.
  - Predictive models limited by data richness and accuracy.
-

## **13. Conclusion**

The project successfully analyzed India's 2020 road accident data and offered valuable insights for regulatory improvements. Combining data science with public policy can lead to smarter, safer roads.